

Supporting Hibernate on FreeBSD

Olivier Certner (o1ce@)

Kumacom SARL

Sponsored by The FreeBSD Foundation

BSDCan 2026

Recent/On-going Work

- FreeBSD Foundation's Laptop Project, Power-related work
 - Collaborative Processor Performance Control (CPPC)
 - Can set a balance between efficiency and performance
 - Add knobs to more finely constrain efficiency/performance
 - In collaboration with ShengYi Hung
 - S0ix
 - Support Aymeric Wibo's work aimed at AMD Framework Laptops
 - `acpi_spmc(4)`: More robust, generic, more diagnostics
 - Aim to make it work on Intel laptops also
 - Hibernate ($\approx$ ACPI S4) $\Rightarrow$ [This talk!](#)
- `mac_do(4)/mdo(1)`
 - Fix and merge “executable paths” (GSoC 2025)
 - Ensure sequential consistency of all jail configuration accesses
- Scheduler
 - Identify ULE's shortcomings, prepare potential fixes/evolutions
 - Plans for performance/battery life on hybrid architectures

This Talk

1 Overview

This Talk

- 1 Overview
- 2 Dumping Memory

This Talk

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap

This Talk

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap
- 4 State Consistency

This Talk

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap
- 4 State Consistency
- 5 Snapshots

Outline

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap
- 4 State Consistency
- 5 Snapshots

Hibernate?



Suspend/Resume

Suspend

- Stop non-system activity
- Save some context (e.g., buffers to disks)
- Stop CPUs
- Power off some devices (or put into low power)

Suspend/Resume

Suspend

- Stop non-system activity
- Save some context (e.g., buffers to disks)
- Stop CPUs
- Power off some devices (or put into low power)

Resume

- Power on
- CPUs resume execution
- Restore context
- Restart activity

Hibernate

Suspend

- Stop non-system activity
- Save **all** context (**memory, devices**)
- Stop CPUs
- Power off **everything**

Resume

- Power on
- CPUs resume execution
- Restore context
- Restart activity

Hibernate

Characteristics

- Maximally conserve power/battery
- Maintain all application contexts
- Better resume latency than full boot

Use Cases

- Save battery when not in use (Laptops)
- Minimize servers/desktops downtime on power off
- Maintain full context for days or more

ACPI States

Sleeping States

- S1 Low-wake latency
- S2 Like S1, with caches and CPU contexts lost
- S3 Most context lost, except for memory
- S4 Devices powered off, platform context saved

ACPI States

Sleeping/Soft-off States

- S1 Low-wake latency
- S2 Like S1, with caches and CPU contexts lost
- S3 Most context lost, except for memory
- S4 Devices powered off, platform context saved
- S5 Soft-off (G2)

ACPI States

System States

- S0 Running (G0) (Processor states: C0, C1, etc.)
- S1 Low-wake latency
- S2 Like S1, with caches and CPU contexts lost
- S3 Most context lost, except for memory
- S4 Devices powered off, platform context saved
- S5 Soft-off (G2)

ACPI States

System States

S0 Running (G0) (Processor states: C0, C1, etc.) **S0ix**

S1 Low-wake latency

S2 Like S1, with caches and CPU contexts lost

S3 Most context lost, except for memory

S4 Devices powered off, platform context saved

S5 Soft-off (G2)

ACPI States

System States

- S0** Running (G0) (Processor states: C0, C1, etc.) **S0ix**
- S1** Low-wake latency
- S2** Like S1, with caches and CPU contexts lost
- S3** Most context lost, except for memory
- S4** Devices powered off, platform context saved $\approx$ **Hibernate**
- S5** Soft-off (G2)

ACPI S4

Saving Memory

- OS-initiated
- Firmware-initiated (“S4BIOS”)

Platform Support

- Presence of _S4
- S4BIOS_F flag in FACS table

S4BIOS

Advantages

- Makes leveraging S4 easier
 - OS does not need to support saving memory
 - Similar to other sleeping states
- Popular from the 2000s to the early 2010s
- Supported by FreeBSD!

Drawbacks

- Not supported anymore by non-too-old machines
- Needs dedicated non-volatile storage
 - Usually, a disk slice/partition

“OS-initiated” Hibernate

This work

- Kernel: Saves state to an hibernate *image* on disk
 - Memory
 - Metadata to re-activate it
- Kernel: Shutdown the machine (ACPI: using S4)
Computer off
- Loader: Loads the image and gives control back to the kernel
- Kernel: Initializes the hardware and resumes the saved state

“OS-initiated” Hibernate

This work

- Kernel: Saves state to an hibernate *image* on disk
 - Memory
 - Metadata to re-activate it
- Kernel: Shutdown the machine (ACPI: using S4)
Computer off
- Loader: Loads the image and gives control back to the kernel
- Kernel: Initializes the hardware and resumes the saved state

Looks simple enough, doesn't it?

Outline

- 1 Overview
- 2 Dumping Memory**
- 3 Bootstrap
- 4 State Consistency
- 5 Snapshots

Existing Dump Infrastructure

- Purpose: Debug
 - On panic
 - On request (e.g., `reboot -d`)
- Different types
 - Full
 - Text
 - Mini (default)
- Writes to a swap partition

Existing Dump Infrastructure

- Purpose: Debug
 - On panic
 - On request (e.g., `reboot -d`)
- Different types
 - Full
 - Text
 - Mini (default)
- Writes to a swap partition

⇒ Let's try to reuse this

Existing Dump Infrastructure

- Mini dumps look appealing
 - Only mapped pages

Existing Dump Infrastructure

- Mini dumps look appealing
 - Only mapped pages
 - But...
 - Only pages from the kernel map
 - Pages marked `PG_NODUMP` are not saved

Existing Dump Infrastructure

- Mini dumps look appealing
 - Only mapped pages
 - But...
 - Only pages from the kernel map
 - Pages marked `PG_NODUMP` are not saved
- Full dumps
 - All memory managed by the kernel

Hibernate Image

Memory

- Physical chunks (`dump_avail[]`)
- Covers all type of states
 - Applications/Userland
 - Kernel
 - Devices?

Hibernate Image

Memory

- Physical chunks (`dump_avail[]`)
- Covers all type of states
 - Applications/Userland
 - Kernel
 - Devices?

Metadata/Bootstrap

- Where to restore memory
- How to bootstrap the kernel
- Others?

Hibernate Image Format

ELF

- Flexible, well documented, supported

Hibernate Image Format

ELF

- Flexible, well documented, supported
- Header metadata
 - Natural width, endianness, machine, etc.
 - Type: `ET_FREEBSD_HIBERNATE_IMAGE`

Hibernate Image Format

ELF

- Flexible, well documented, supported
- Header metadata
 - Natural width, endianness, machine, etc.
 - Type: ET_FREEBSD_HIBERNATE_IMAGE
- Multiple typed segments
 - One per memory chunk (PT_LOAD)
 - Size, physical address
 - Ad-hoc segments for additional metadata

Outline

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap**
- 4 State Consistency
- 5 Snapshots

Loader

- Worked on by Konstantin Belousov (kib@)
- EFI program
- Load the image, pass control to the kernel

Image loading

- Put memory chunks (ELF segments) at their physical address

Image loading

- Put memory chunks (ELF segments) at their physical address
- Collisions!

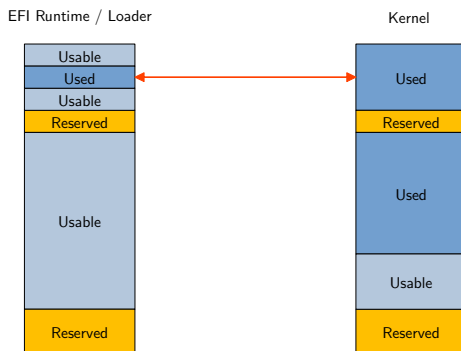


Image loading

- Put memory chunks (ELF segments) at their physical address
- Trampolines!

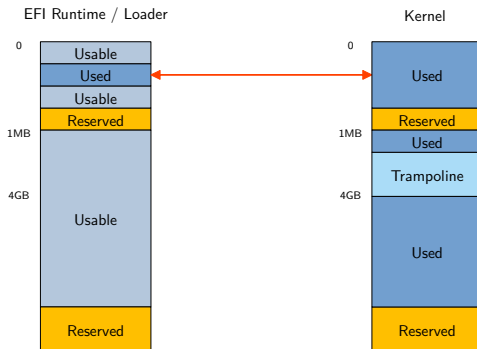


Image loading

- Put memory chunks (ELF segments) at their physical address
- Trampolines!

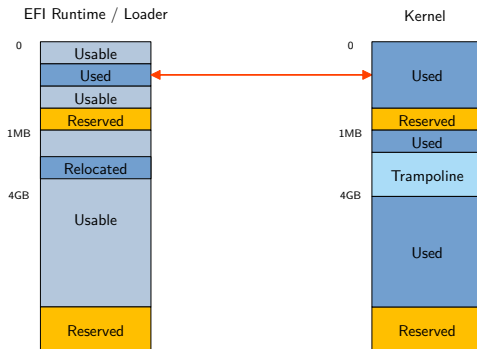
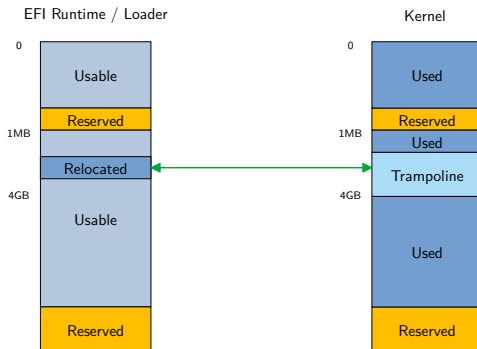


Image loading

- Put memory chunks (ELF segments) at their physical address
- Trampolines!



Bootstrap Metadata

- Where to restore memory
 - ELF segments

Bootstrap Metadata

- Where to restore memory
 - ELF segments
- Trampoline
 - Pointers to kernel-allocated pages
 - Must cover the loader's relocated part
 - Specific physical address placement

Bootstrap Metadata

- Where to restore memory
 - ELF segments
- Trampoline
 - Pointers to kernel-allocated pages
 - Must cover the loader's relocated part
 - Specific physical address placement
- How to bootstrap the kernel
 - Establish the kernel map
 - Entry point

Kernel Entry Point

Minimal Interface

- To avoid having to update the loader on kernel changes
- System registers (CR0, CR4, CR3)
- Instruction pointer (RIP), stack pointer (RSP)

Kernel Entry Point

Minimal Interface

- To avoid having to update the loader on kernel changes
- System registers (CR0, CR4, CR3)
- Instruction pointer (RIP), stack pointer (RSP)

Kernel Context is Bigger

- MSRs (EFER, KERNEL_GS_BASE, STAR, LSTAR, etc.)
- General-purpose registers
- FPU, SSE states

Kernel Context Restore

Stage 1 Loader jumps to kernel's entry point

Stage 2 Kernel loads full context

Kernel Context Restore

Stage 1 Loader jumps to kernel's entry point

Stage 2 Kernel loads full context

⇒ Kernel needs an additional argument to grab that context

Outline

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap
- 4 State Consistency**
- 5 Snapshots

Constraints

- Restored system must be viable
 - It must continue as if hibernate did not occur
 - No external discrepancies (storage data, devices state)
 - No internal ones (VM, malloc, threads, page tables, etc.)

Constraints

- Restored system must be viable
 - It must continue as if hibernate did not occur
 - No external discrepancies (storage data, devices state)
 - No internal ones (VM, malloc, threads, page tables, etc.)
- Restoration failure not to affect storage data
 - Also, storage data up-to-date

Moving States

- Applications
- Devices
- Kernel

Problem: Concurrent changes?

Moving States

- Applications $\Rightarrow$ Stop all processes/threads
- Devices
- Kernel

Problem: Concurrent changes?

Moving States

- Applications $\Rightarrow$ Stop all processes/threads
- Devices $\Rightarrow$ Suspend them
- Kernel

Problem: Concurrent changes?

Moving States

- Applications $\Rightarrow$ Stop all processes/threads
- Devices $\Rightarrow$ Suspend them, not that simple...
- Kernel $\Rightarrow$ Consistent state view while saving?

Problem: Concurrent changes?

Kernel High-Level State

System processes/threads

- A priori not stoppable/resumable at will
- Must drain any pending disk requests
 - No new ones (filesystem suspension)
 - Finish existing ones (GEOM)
- Suspend all other CPUs, stop scheduler
 - Needed by existing dump routines

Devices

Suspend vs. Dumping

- Call `DEVICE_SUSPEND()` on the root bus

Devices

Suspend vs. Dumping

- Call `DEVICE_SUSPEND()` on the root bus
- But now, how to dump?

Devices

Suspend vs. Dumping

- Call `DEVICE_SUSPEND()` on the root bus
- But now, how to dump?
- Resume only the disk

Devices

Suspend vs. Dumping

- Call `DEVICE_SUSPEND()` on the root bus
- But now, how to dump?
- Resume only the disk $\Rightarrow$ Complications

Devices

Problems

- Requires powering up part of the device tree
 - Bridges
 - Power resources

Devices

Problems

- Requires powering up part of the device tree
 - Bridges
 - Power resources
- Some drivers are not drivers(9)
 - `nda(4)`, `ada(4)`, `da(4)` (CAM-based)
 - No `DEVICE_SUSPEND()`/`DEVICE_RESUME()`
 - Some event handlers instead (`EVENTHANDLER(9)`)

Devices

Problems

- Requires powering up part of the device tree
 - Bridges
 - Power resources
- Some drivers are not drivers(9)
 - `nda(4)`, `ada(4)`, `da(4)` (CAM-based)
 - No `DEVICE_SUSPEND()`/`DEVICE_RESUME()`
 - Some event handlers instead (`EVENTHANDLER(9)`)
- State modifications
 - DMA needed, even for polled I/O
 - Allocations while dumping
 - IOMMU, bounce buffers

Devices

Possible Solutions

- Infrastructure for selective device suspend/resume

Devices

Possible Solutions

- Infrastructure for selective device suspend/resume
 - `DEVICE_QUIESCE()` exists, but no implementation

Devices

Possible Solutions

- Infrastructure for selective device suspend/resume
 - `DEVICE_QUIESCE()` exists, but no implementation
- Still, what about memory allocations?

Outline

- 1 Overview
- 2 Dumping Memory
- 3 Bootstrap
- 4 State Consistency
- 5 Snapshots**

Memory Snapshots

Concept

Take a **snapshot** after quiescing and before dump

Memory Snapshots

Concept

Take a **snapshot** after quiescing and before dump

Benefits

No more constraints on dump!

- No need to stop CPUs, scheduler (except during the snapshot)
- Memory allocation at will
- Dump to files
- Easier encrypted dumps

“A Priori” Variant

Snapshot all memory from the start

“A Priori” Variant

Snapshot all memory from the start

- Pages to copy must not exceed about half of physical memory
- Get rid of (part of) caches to make room
- Do not copy unnecessary pages (e.g., userland)

“Lazy” Variant

Make pages read-only, snapshot on write

“Lazy” Variant

Make pages read-only, snapshot on write

- Walk kernel page table, make pages read-only
- On fault, snapshot the page, restore access bits, continue
- Interrupts supported through preallocated pages

Common Traits

- Determine the pages that need copying

Common Traits

- Determine the pages that need copying
- Allocate memory to copy them

Common Traits

- Determine the pages that need copying
- Allocate memory to copy them
- Maintain a mapping between original and copies

Common Traits

- Determine the pages that need copying
- Allocate memory to copy them
- Maintain a mapping between original and copies
- On dump, the copies go onto the disk

Status

- Early dump prototype, produces an (inconsistent) image
- Work on-going on the EFI loader (stage 1; kib@)
- Code sketch for kernel bootstrap (stage 2)
- Start working on quiescing high-level kernel state
- Finish analysis and decide on the consistency approach

Thanks!

To Warner Losh (`imp@`) and Konstantin Belousov (`kib@`)

Questions? Thoughts?

olce@

Sponsored by The FreeBSD Foundation